

AIFA — Autonomous Intelligent Factory Agent

Technical System Specification (v3)

Repository-centric · Human-in-the-loop · Multi-agent · Git-first

Tài liệu: Technical System Specification — không phải mô tả UI

Phạm vi: Kiến trúc hệ thống, luồng nghiệp vụ, hợp đồng UI, state machine, xử lý lỗi

Người biên soạn: Lê Tú Nam (Namlee)

Phiên bản: v3

Mục lục

- 1. Vision
- 2. System Architecture
- 3. Repository Workflow
- 4. Session Lifecycle
- 5. Agent Pipeline (ARCH · PO · UX · DEV · QA)
- 6. Human Interaction
- 7. Git Workflow
- 8. Agents Task Page
- 9. Runtime State
- 10. Event Flow
- 11. Git Commit Policy
- 12. Final Review
- 13. final.md
- 14. UI Contract
- 15. State Machine
- 16. Sequence Diagram
- 17. Error Handling

CHƯƠNG 1

Vision

AIFA (Autonomous Intelligent Factory Agent) là một Autonomous Software Factory: một hệ thống multi-agent vận hành trực tiếp trên repository của người dùng, thực thi từng giai đoạn của SDLC (kiến trúc → sản phẩm → thiết kế → lập trình → kiểm thử) thông qua các phiên Claude Code có vai trò luân chuyển, dưới sự giám sát và phê duyệt liên tục của con người.

Bốn nguyên lý nền tảng:

- Repository-centric architecture — repository Git là nguồn sự thật duy nhất; mọi output của agent đều tồn tại dưới dạng file/commit trong repo, không có trạng thái nghiệp vụ nằm ngoài Git.
- Human-in-the-loop orchestration — không có bước nào được tự động bỏ qua sự phê duyệt của con người; mỗi agent có quyền hỏi (Human Question) và mỗi output đều phải qua Human Review.
- Multi-agent collaboration — 5 agent chuyên biệt (ARCH, PO, UX, DEV, QA) chạy tuần tự trong cùng một run, giao tiếp qua Agent-to-Agent (A2A) handoff dựa trên artifact đã commit.
- Git-first workflow — mọi hành động approve đều kéo theo write file → git add → commit → push; lịch sử Git chính là audit trail của hệ thống.

Khác với một trình sinh code tự động, AIFA không tối ưu cho tốc độ tối đa mà tối ưu cho khả năng kiểm soát và truy vết: mỗi quyết định của agent đều có thể bị chặn, chất vấn, hoặc từ chối bởi người vận hành trước khi đi vào lịch sử Git.

CHƯƠNG 2

System Architecture

Hệ thống được chia thành 6 lớp độc lập, giao tiếp qua các hợp đồng dữ liệu rõ ràng.

2.1 Overall Architecture



2.2 Backend

- Node.js + Express, REST cho các hành động điều khiển (start run, approve, answer question).
- Prisma ORM trên SQLite — lưu Run, AgentEvent, GateDecision, Session.
- Không giữ trạng thái nghiệp vụ chỉ trong RAM — mọi state quan trọng được persist trước khi phát event.

2.3 Frontend

- React + Vite + TypeScript, một Workflow Store duy nhất (single source of truth phía client).
- Không tự poll dữ liệu — toàn bộ cập nhật đến qua SSE stream, store chỉ phản ứng với event.

2.4 Agent Runtime

- Một session Claude Code CLI cho mỗi run; vai trò (ARCH/PO/UX/DEV/QA) chuyển đổi bằng role-switching prompt, không khởi tạo lại session giữa các agent trong cùng run.

- Agent Adapter Contract chuẩn hoá input/output giữa Runtime và Backend, độc lập với model provider (Claude / MiniMax / OpenRouter alias).

2.5 Git Layer

- Mỗi run có một working branch riêng, clone/workspace cách ly (per-run Git isolation), tránh xung đột khi nhiều run chạy song song (parallel run support).

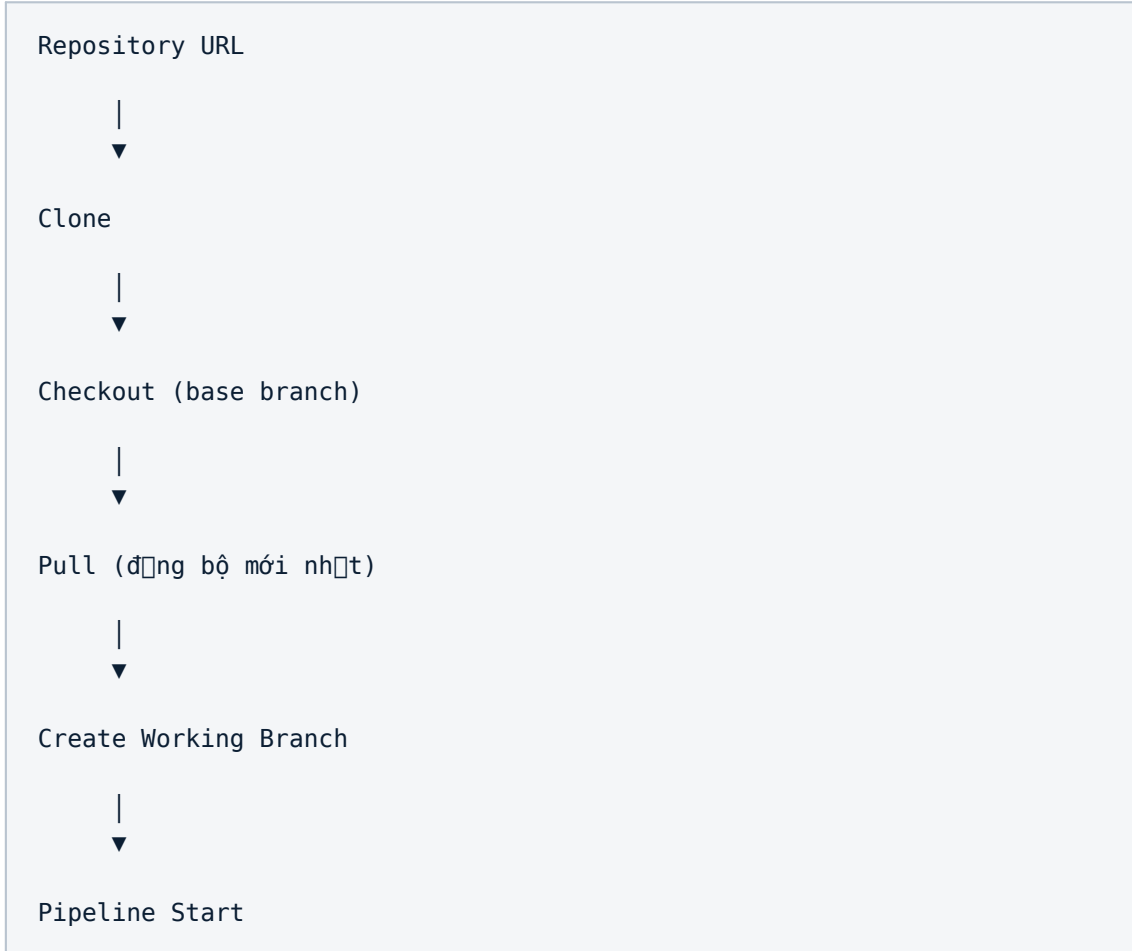
2.6 SSE Layer & State Management

- SSE là kênh truyền event một chiều Backend → Frontend, không dùng polling, không có store trùng lặp.
- State Management tuân theo state machine tường minh ở Chương 15, mọi transition đều được ghi lại thành AgentEvent chuẩn hoá.

CHƯƠNG 3

Repository Workflow

Trước khi Pipeline bắt đầu, hệ thống phải chuẩn bị một workspace Git sạch, cách ly cho run hiện tại.



Ghi chú vận hành:

- Repository URL — do người dùng cung cấp khi tạo run; hệ thống validate quyền truy cập trước khi clone.
- Base Branch — nhánh nguồn (thường là main/develop); Working Branch luôn được tạo mới từ base branch sau khi pull, đảm bảo không có drift.
- Workspace — mỗi run có một thư mục workspace riêng trên disk (per-run isolation), bị dọn dẹp sau khi run kết thúc hoặc bị hủy.
- Nếu Clone/Checkout/Pull thất bại, run không được phép chuyển sang trạng thái Preparing Repository → Running (xem Chương 17 — Error Handling: Git Failure).

CHƯƠNG 4

Session Lifecycle

Lifecycle của một run, từ khi khởi tạo đến khi hoàn tất, tuần hoàn qua từng agent trong Pipeline.

Idle



Preparing Repository



Running



Pending Gate



Review



Commit



A2A (Agent-to-Agent handoff)



Next Agent



Final Review



Generate final.md



Completed

Vòng Running → Pending Gate → Review → Commit → A2A → Next Agent lặp lại cho từng agent trong Pipeline (Chương 5). Chỉ sau khi agent cuối cùng (QA) hoàn tất, hệ thống mới chuyển sang Final Review (Chương 12).

CHƯƠNG 5

Agent Pipeline

Pipeline gồm 5 agent chạy tuần tự trong một run: ARCH → PO → UX → DEV → QA. Mỗi agent tuân theo cùng một khung 9 mục dưới đây, chỉ khác nhau về nội dung chuyên môn.

ARCH — Architecture Agent

Trường	Nội dung
Responsibilities	Phân tích yêu cầu, đề xuất kiến trúc hệ thống, xác định ràng buộc kỹ thuật và rủi ro.
Input	Feature Request gốc từ người dùng + toàn bộ context repository hiện có.
Output	architecture.md — kiến trúc đề xuất, các quyết định thiết kế, trade-off.
Runtime	Claude Code session ở vai trò ARCH; đọc repo, không chỉnh sửa code.
Human Question	Có quyền hỏi khi yêu cầu mơ hồ hoặc thiếu ràng buộc kỹ thuật.
Human Review	Bắt buộc trước khi commit architecture.md.
Output Review	Markdown render trong Output Review panel.
Commit	commit riêng cho architecture.md, message theo chuẩn Chương 7.
Transition	A2A → PO, kèm architecture.md làm input.

PO — Product Owner Agent

Trường	Nội dung
Responsibilities	Chuyển kiến trúc thành yêu cầu sản phẩm cụ thể: user story, acceptance criteria, phạm vi.
Input	architecture.md + Feature Request gốc.
Output	product-spec.md — user stories, acceptance criteria, out-of-scope.
Runtime	Claude Code session ở vai trò PO.
Human Question	Câu hỏi về phạm vi, độ ưu tiên, ràng buộc nghiệp vụ.
Human Review	Bắt buộc trước khi commit product-spec.md.
Output Review	Markdown; có thể kèm bảng acceptance criteria.
Commit	commit riêng cho product-spec.md.

Transition	A2A → UX, kèm product-spec.md.
------------	--------------------------------

UX — UX/Design Agent

Trường	Nội dung
Responsibilities	Thiết kế luồng người dùng, wireframe, UI contract sơ bộ cho các tính năng trong spec.
Input	product-spec.md + architecture.md.
Output	ux-design.md (+ ảnh wireframe nếu có) — luồng, UI contract, trạng thái UI.
Runtime	Claude Code session ở vai trò UX.
Human Question	Câu hỏi về hành vi UI, trạng thái edge-case.
Human Review	Bắt buộc; Output Review hiển thị cả markdown và image preview.
Output Review	Markdown + image preview (nếu có wireframe).
Commit	commit riêng cho ux-design.md và assets liên quan.
Transition	A2A → DEV, kèm ux-design.md.

DEV — Development Agent

Trường	Nội dung
Responsibilities	Triển khai code theo architecture.md, product-spec.md, ux-design.md.
Input	Toàn bộ artifact của ARCH/PO/UX + codebase hiện tại.
Output	Code diff thực tế (deferred-apply search/replace blocks) trên working branch.
Runtime	Claude Code session ở vai trò DEV; có quyền đọc/viết file trong workspace.
Human Question	Câu hỏi khi phát hiện xung đột giữa spec và code hiện tại.
Human Review	Bắt buộc — diff phải được duyệt trước khi apply và commit.
Output Review	Code diff view; không auto-apply nếu chưa approve.
Commit	commit theo từng nhóm thay đổi logic, message mô tả rõ thay đổi.
Transition	A2A → QA, kèm danh sách file đã thay đổi.

QA — Quality Assurance Agent

Trường	Nội dung
Responsibilities	Kiểm thử tĩnh (static-only QA): review code, kiểm tra tuân thủ spec, phát hiện lỗi rõ ràng.
Input	Code diff của DEV + toàn bộ spec ARCH/PO/UX.
Output	qa-report.md — danh sách vấn đề, mức độ nghiêm trọng, đề xuất khắc phục.
Runtime	Claude Code session ở vai trò QA; không thực thi test suite động.
Human Question	Câu hỏi khi mức độ nghiêm trọng của một vấn đề cần quyết định của người dùng.
Human Review	Bắt buộc trước khi commit qa-report.md.
Output Review	Markdown report, có thể tham chiếu chéo tới diff của DEV.
Commit	commit riêng cho qa-report.md.
Transition	Kết thúc Pipeline → Final Review (Chương 12).

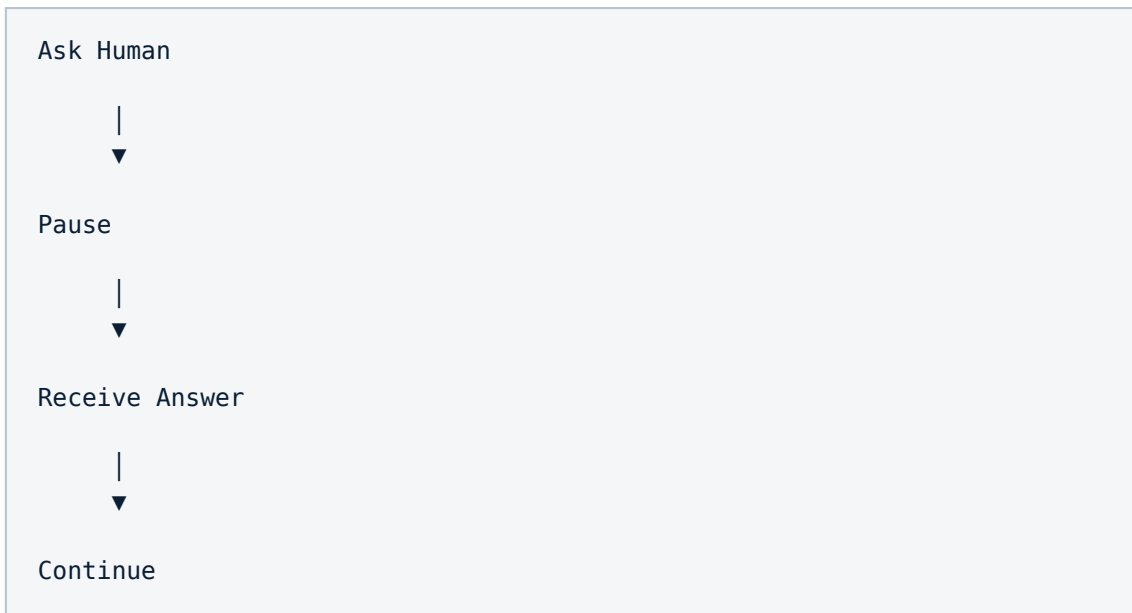
CHƯƠNG 6

Human Interaction

Đây là chương quan trọng nhất của spec: AIFA không phải một pipeline tự động chạy xuyên suốt, mà là một hệ thống mà con người luôn ở trong vòng lặp quyết định.

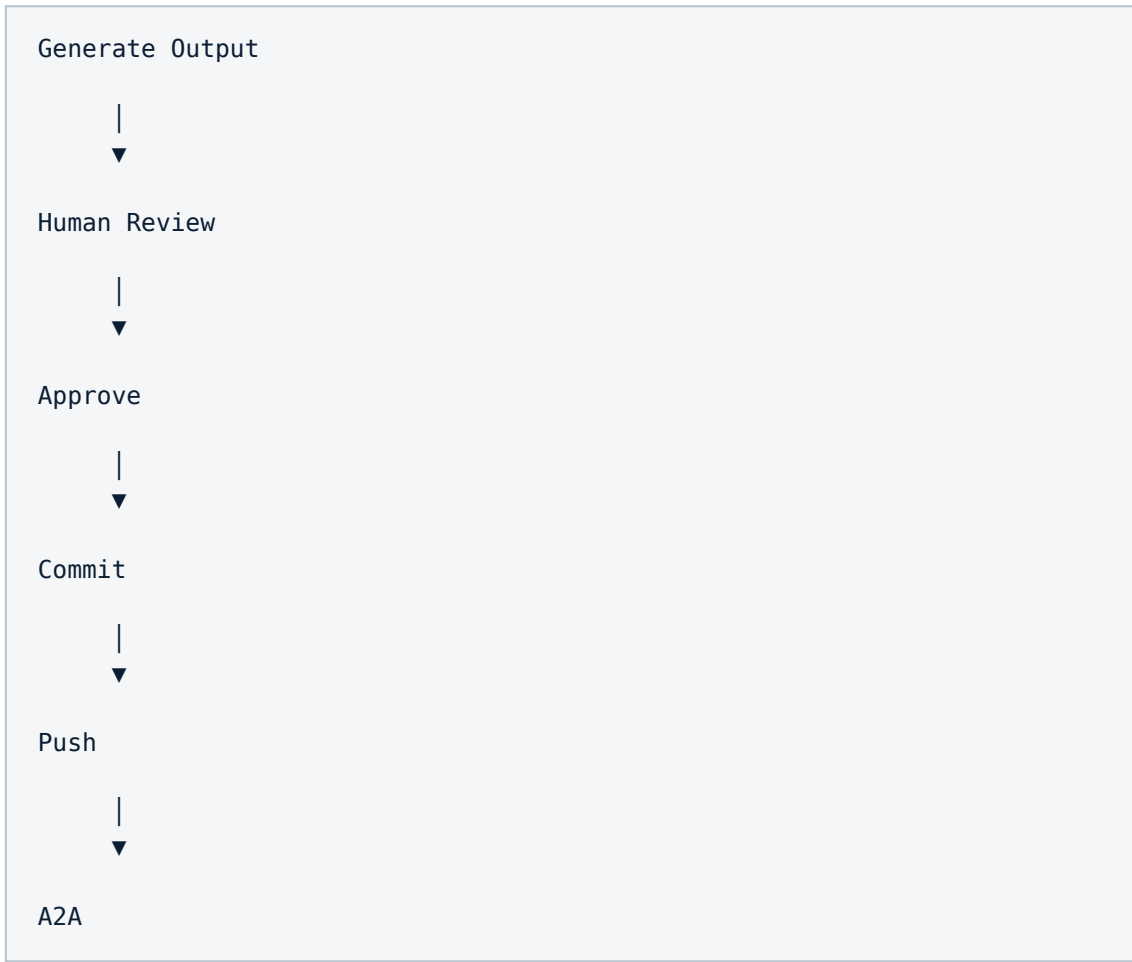
6.1 Human Question (Mandatory)

Mọi agent — không riêng PO — đều có quyền dừng lại và hỏi người dùng khi gặp thông tin mơ hồ hoặc thiếu ràng buộc. Đây là đặc trưng khác biệt của AIFA so với pipeline agent tuyến tính thông thường.



6.2 Human Review (Mandatory)

Sau khi một agent hoàn thành output, hệ thống luôn dừng ở trạng thái chờ duyệt. Không có bước "auto-skip" nào được phép tồn tại trong hệ thống.



6.3 Output Review

Output Review hiển thị đầy đủ nội dung output của agent ở một nơi duy nhất, bao gồm:

- Markdown render (spec, report).
- Code diff (đối với output của DEV).
- HTML preview (khi output có thành phần UI render được).
- Image preview (wireframe, sơ đồ, nếu agent tạo ra).

Artifact không còn là một tab riêng. Artifact được nhúng trực tiếp trong Output Review — người dùng xem toàn bộ kết quả của agent trong một luồng liên tục, không phải chuyển tab để đối chiếu.

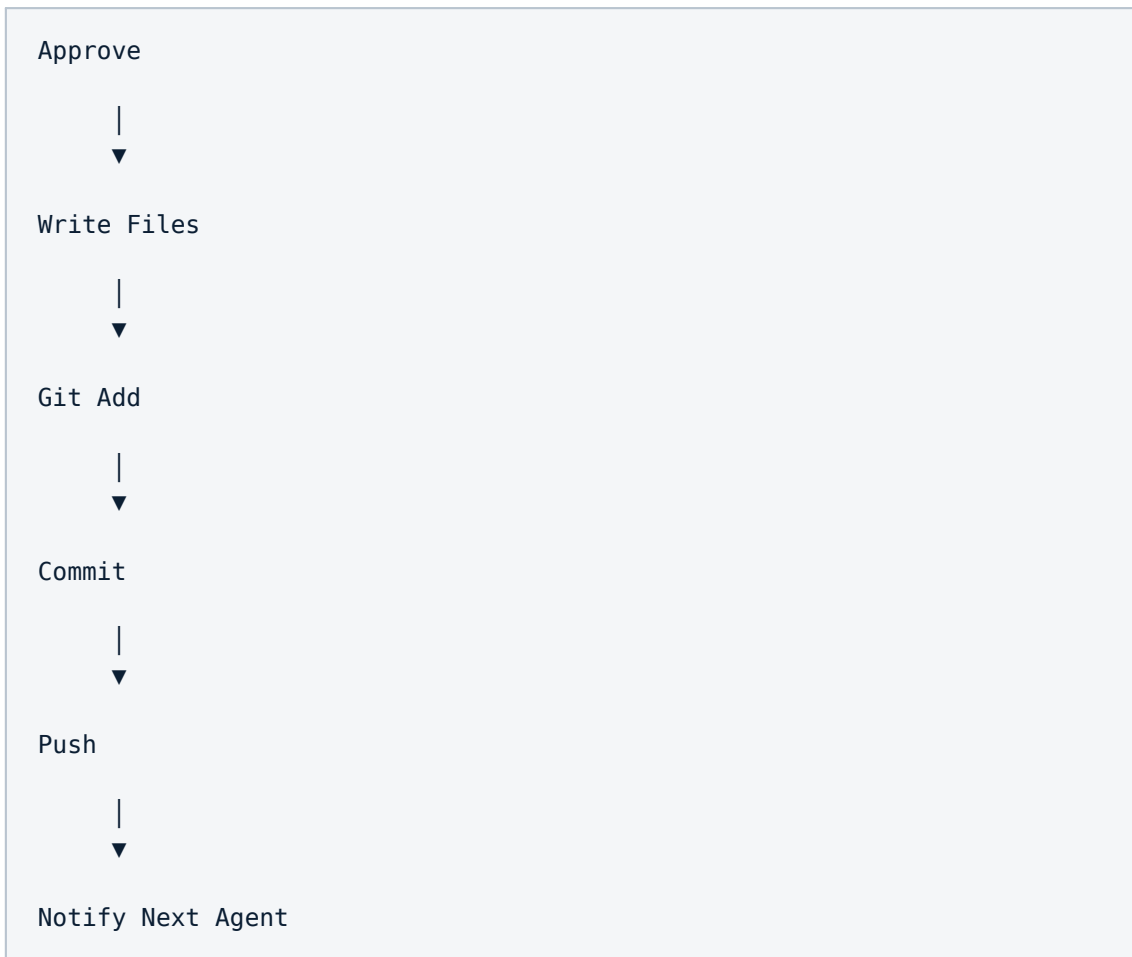
Git Workflow

7.1 Quy tắc chung

- Branch creation — một working branch cho mỗi run, tạo từ base branch sau Pull (Chương 3).
- Commit policy — commit theo từng agent, không dồn thành một commit cuối pipeline (chi tiết Chương 11).
- Commit message — chuẩn hoá theo mẫu: [agent][run-id] mô tả ngắn thay đổi.
- Push policy — push ngay sau mỗi commit thành công, không giữ commit chỉ ở local.

7.2 Approve không chỉ là approve

Hành động Approve của người dùng trong Human Review kích hoạt một chuỗi hành động Git tự động, không thể tách rời:



Nếu bất kỳ bước nào trong chuỗi trên thất bại (*Write Files*, *Git Add*, *Commit*, *Push*), toàn bộ hành động Approve được coi là chưa hoàn tất và hệ thống phải báo lỗi rõ ràng cho người dùng (xem Chương 17).

Agents Task Page

Đây là spec giao diện lớn nhất trong tài liệu — trang trung tâm nơi người dùng theo dõi và tương tác với toàn bộ run đang chạy.

8.1 Session Summary

Hiển thị thông tin tổng quan của run:

- Repository
- Branch
- Commit SHA hiện tại
- Pipeline Status
- Current Agent

Kèm dashboard trạng thái:

- Completed Agents
- Generated Files
- Errors
- Warnings

8.2 Runtime

Hiển thị hoạt động thời gian thực của agent đang chạy:

- Current Step
- Current Action
- Current File
- Runtime Log

Không còn Audit page riêng. Runtime Log chính là timeline duy nhất — mọi AgentEvent (bắt đầu step, gọi tool, ghi file, lỗi, hoàn tất) đều xuất hiện tuần tự tại đây.

8.3 Human Question

Panel này luôn hiển thị (không ẩn ngấm) mỗi khi agent hiện tại phát sinh câu hỏi cần con người trả lời; Runtime bị pause cho tới khi có câu trả lời (xem 6.1).

8.4 Human Review

Luôn hiển thị ngay sau khi một agent hoàn thành output, trước khi chuyển sang agent kế tiếp.

8.5 Output Review

Hiển thị đầy đủ output của agent theo đúng định dạng ở 6.3. Không còn tab Artifact riêng — Artifact được render trực tiếp trong Output Review.

CHƯƠNG 9

Runtime State

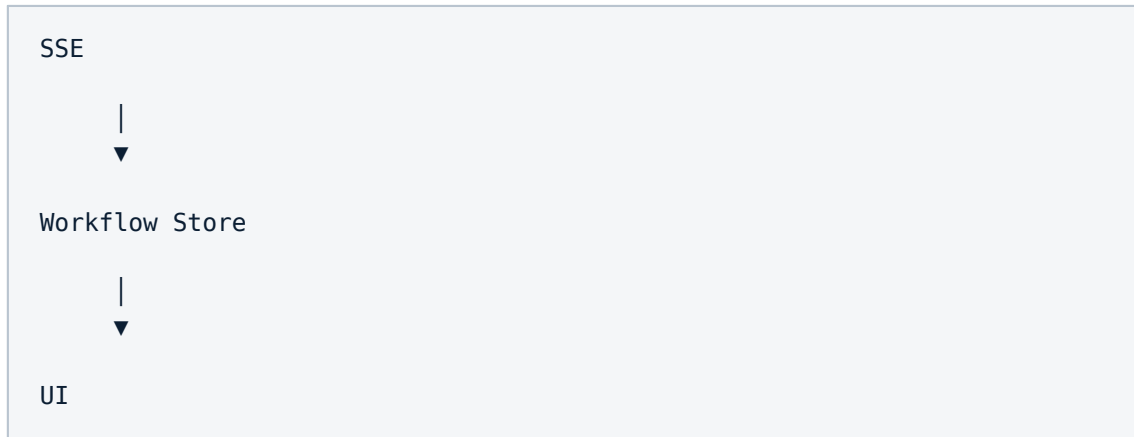
Runtime State (Session State) là Single Source of Truth duy nhất cho toàn bộ trạng thái của một run.

- Không polling — Frontend không tự gọi lại API để refresh trạng thái.
- Không duplicate store — chỉ một Workflow Store phía Frontend, được đồng bộ hoàn toàn từ SSE event; không có store thứ hai giữ bản sao trạng thái khác nhau.
- Session State phía Backend được persist qua Prisma trước khi phát event — Frontend reload không mất trạng thái.

CHƯƠNG 10

Event Flow

Toàn bộ cập nhật trạng thái trong hệ thống đi theo một chiều duy nhất:



Backend phát AgentEvent chuẩn hoá qua SSE → Workflow Store cập nhật state → UI re-render theo state. Không có chiều ngược (UI không tự suy luận trạng thái ngoài những gì Store cung cấp).

Git Commit Policy

Write Output



Git Add



Commit

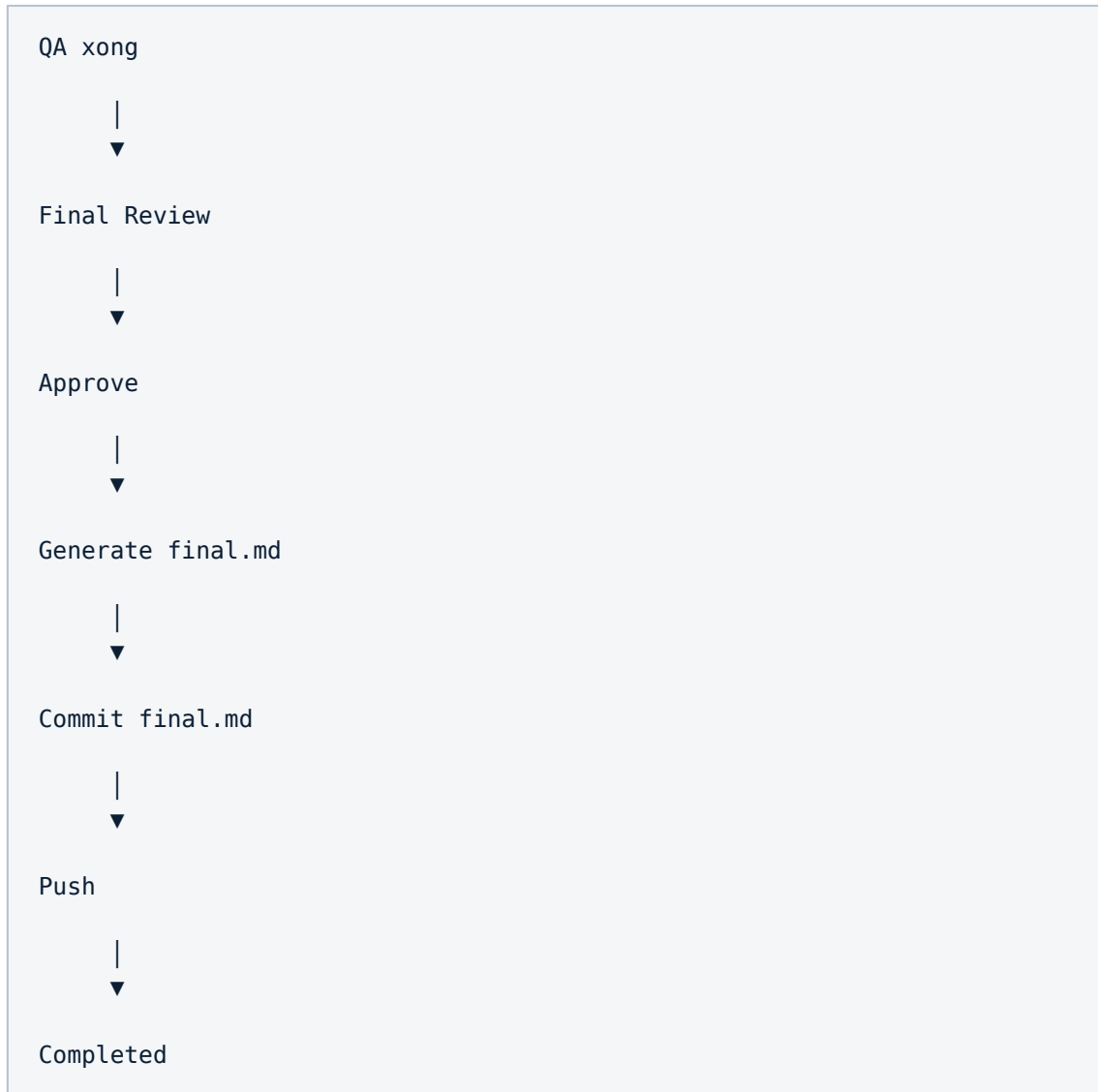


Push

Mỗi lần Approve đều tạo commit riêng. Không commit dồn ở cuối pipeline — mỗi agent tạo ra ít nhất một commit độc lập tại thời điểm output của agent đó được approve, giúp lịch sử Git phản ánh chính xác tiến trình của từng giai đoạn SDLC.

Final Review

Sau khi QA hoàn tất, run chuyển sang giai đoạn tổng kết:



Final Review là điểm phê duyệt cuối cùng của con người trước khi run được đóng lại; sau bước này, run không thể quay lại trạng thái Running.

CHƯƠNG 13

final.md

final.md là artifact tổng hợp duy nhất của toàn bộ run, đóng vai trò báo cáo bàn giao. Cấu trúc bắt buộc:

Mục	Nội dung
Repository	URL repo, base branch, working branch, commit SHA cuối cùng.
Feature Request	Yêu cầu gốc do người dùng nhập khi tạo run.
ARCH Output	Toàn văn architecture.md.
PO Output	Toàn văn product-spec.md.
UX Output	Toàn văn ux-design.md.
DEV Output	Tóm tắt diff, danh sách file thay đổi.
QA Output	Toàn văn qa-report.md.
Audit Trail	Danh sách AgentEvent quan trọng theo timeline.
Human Decisions	Toàn bộ câu hỏi/trả lời và quyết định approve/reject.
Pipeline Summary	Thời gian chạy, số lượng commit, trạng thái cuối cùng.

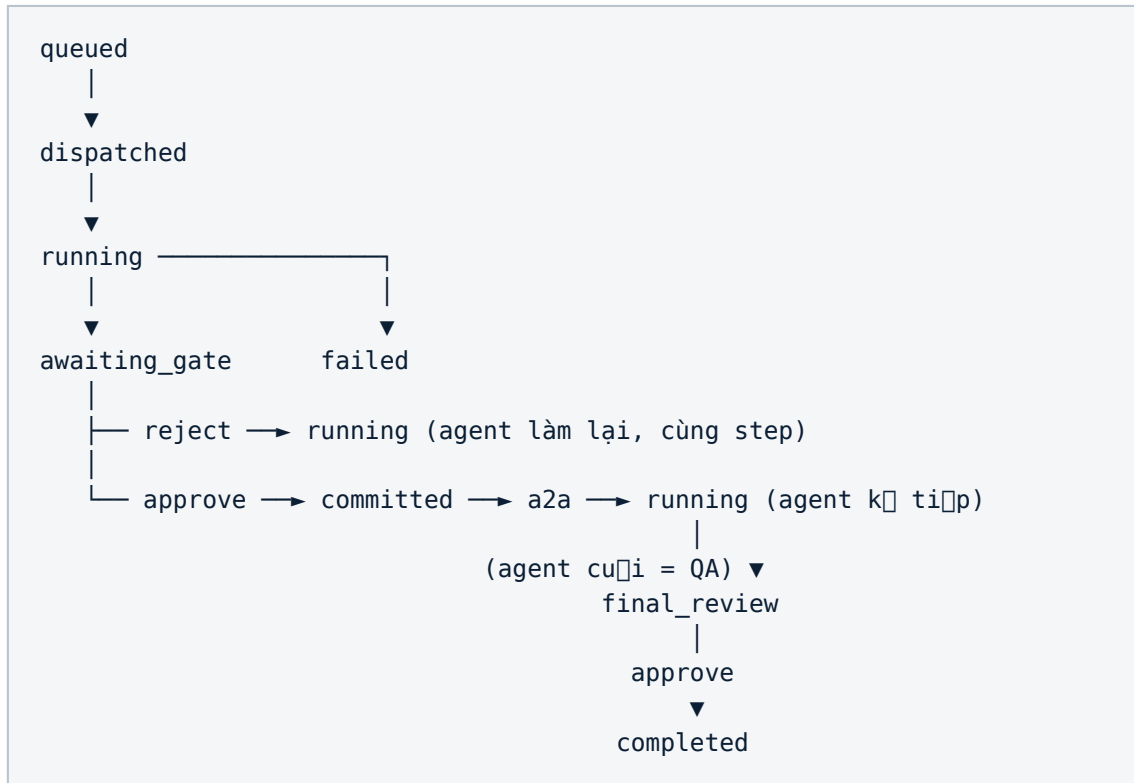
UI Contract

Mỗi component chính trên Agents Task Page phải được định nghĩa đầy đủ props / state / events / data source, không chỉ là wireframe.

Component	Props	State	Events	Data Source
SessionSummaryPanel	runId	repo, branch, sha, status, currentAgent		Workflow Store
RuntimePanel	runId, agentId	currentStep, currentActionLogApprentice	onToggleApprentice	FSSE log Workflow Store
HumanQuestionPanel	question, agentId	answerDraft, isOpen	onSubmitAnswer	AgentEvent(type=QUESTION)
HumanReviewPanel	outputRef, agentId	decision (pending/approved/rejected)	onApprove, onReject	Decision
OutputReviewPanel	outputRef	activeView (markdown/visual/change)	onViewChange	fact + AgentEvent
PipelineDashboard	runId	completedAgents[], generatedFiles[]		AgentEvent streams (Derived)

State Machine

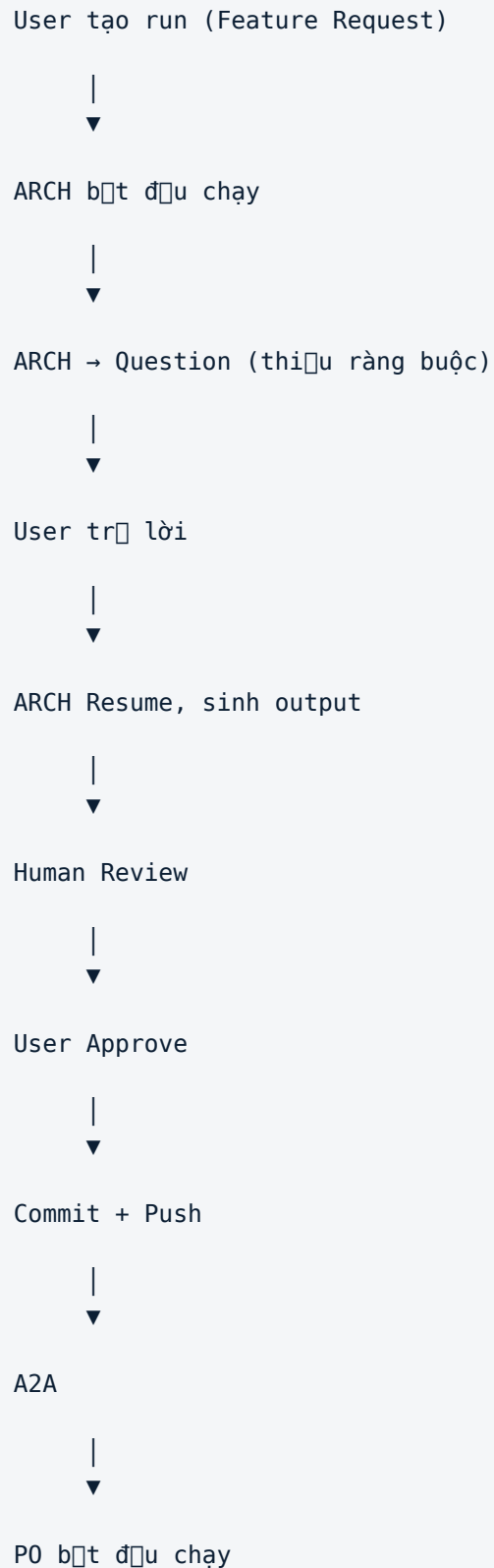
State machine cấp Run (áp dụng cho toàn bộ lifecycle ở Chương 4):



Bất kỳ trạng thái nào (trừ completed và failed) đều có thể chuyển sang failed khi gặp lỗi không phục hồi được (xem Chương 17). GateDecision được persist vào DB tại mọi lần chuyển từ awaiting_gate, không giữ ở bộ nhớ tạm.

Sequence Diagram

Ví dụ minh hoạ một lượt tương tác đầy đủ giữa User và agent ARCH:



Mỗi agent trong Pipeline lập lại đúng cấu trúc sequence trên (có thể bỏ qua bước Question nếu agent không cần hỏi), cho tới khi QA hoàn tất và chuyển sang Final Review.

CHƯƠNG 17

Error Handling

Mọi lỗi trong hệ thống phải được phân loại rõ ràng, hiển thị cho người dùng qua Runtime Log / Dashboard (Chương 8), và có chiến lược retry tương ứng.

Loại lỗi	Nguyên nhân điển hình	Chiến lược xử lý
Git Failure	Clone/Checkout/Pull thất bại (quyền truy cập, xung đột, branch không tồn tại)	Chỉ áp dụng cho pull; không retry; hiển thị lỗi; cho phép retry thủ công
Commit Failure	Xung đột file, working tree bẩn, hook từ chối commit	Giải quyết xung đột, reset hoặc checkout; không retry; hiển thị lỗi; không cho phép retry tự động
Push Failure	Remote reject (non-fast-forward), mất quyền push	Retry một lần; nếu vẫn thất bại, chuyển sang merge conflict; không retry liên tục
Claude Timeout	Session Claude Code không phản hồi trong thời gian chờ	Retry ngay lập tức; nếu vẫn thất bại, chuyển sang Agent khác; không retry liên tục
SSE Disconnect	Mất kết nối stream giữa Frontend và Backend	Backend tự reconnect với backoff; khi reconnect, resync state; Frontend retry ngay lập tức
Retry Strategy (chung)		Exponential backoff cho lỗi network/transient; không tự động retry cho lỗi vĩnh viễn